



DIPARTIMENTO DI **I**NGEGNERIA **E**LETTRICA E **T**ECNOLOGIE DELL'**I**NFORMAZIONE
SCUOLA POLITECNICA E DELLE SCIENZE DI BASE

LAUREA IN INGEGNERIA DELL'AUTOMAZIONE

Hierarchical VLMs Planning and Control with Ambiguity resolution

Candidato:
Vincenzo Palomba

Relatori:
Prof. Alberto Finzi

Academic Year 2025/2026

Outline

1. Planning Layer
2. Executive Layer

Neuro-Symbolic Planning architecture

VLM Architectures

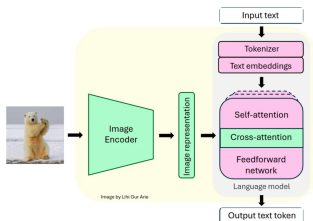


Figure: llama3 VLM architecture

Image + Text $\longrightarrow$ VLM $\longrightarrow$ Text Output

Language Encoder (Transformer)

- ▶ Encoders transform sequences into numerical representation called **embeddings**.
- ▶ A **self-attention mechanism** allows focus on important tokens.
- ▶ **Decoders** output the most statistically probable output.

Vision Encoder (CNN / ViT)

- ▶ Typically utilizes Convolutional Neural Networks (CNNs) or Visual Transformers (ViTs).
- ▶ A ViT processes an image into patches and treats them as sequences, akin to tokens.

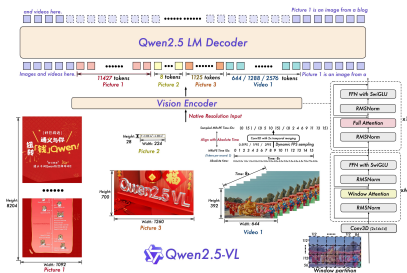
VLM Training methodologies

It is the mechanism that allow fusing information from both vision and language encoders so to correlate images with text and make decision on the 2 modalities together:

- ▶ Contrastive learning: uses joint or shared embedding space. The VLM is trained on datasets of image-text pairs and learns to minimize the distance between embeddings of matching pairs and maximize it for nonmatching pairs.
- ▶ Masking: models learn to predict randomly obscured parts of an input text or image.
- ▶ Generative model training: entails learning to generate new data
- ▶ Pretrained models: A pretrained LLM and a pretrained vision encoder can be used, with an added mapping network layer that aligns or projects the visual representation of an image to the LLM's input space.

VLM Architecture

- ▶ Qwen2.5 LM Decoder
- ▶ ViT:3D convolution to split input visual data into a series of 14×14 patches
- ▶ process video info: preprocessor for dynamic image size adjustment, dynamic frame extraction



Why this choice?

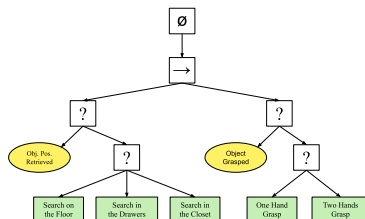
Ambiguity resolution

PDDL planning

PDDL problem generation experiments

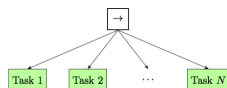
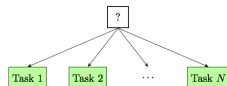
Behavior Trees

- ▶ A **Behavior Tree** is a directed tree with nodes classified as root, control flow, or execution (tasks).
- ▶ Execution starts from the root, which sends *ticks* to its children at a set frequency.
- ▶ A node returns one of three statuses: **Running**, **Success**, or **Failure**.



Control Flow Nodes

- ▶ **Selector:** A fallback node that executes the first child that does not fail. Returns **Success** or **Running**.
- ▶ **Sequence:** Executes children in order until one succeeds. Returns **Failure** or **Running** immediately if a child fails or is still running.



How to build Behavior Trees: *py_trees* library

py_trees is a Python library used to construct Behavior Trees efficiently. It relies primarily on two base classes: **Behaviours** and **Composites**.

- ▶ **Behaviour (Leaf):** The smallest element in a tree. Behaviours typically represent a **condition** (e.g., *Am I hungry?*) or an **action** (e.g., *Buy chocolate cookies*).
- ▶ **Composites (Control Flow):** Responsible for directing the path traced through the tree during a tick. They act as the factories and decision-makers of the tree.
 - ▶ **Selectors:** Fallback decision-makers.
 - ▶ **Sequences:** Order-dependent factories.
 - ▶ **Parallels:** Concurrent execution factories.

BT Builder: Parsing & Sequence Compilation

1. Parsing the Plan

(parse_pddl_action) Cleans raw planner text into structured Python data:

- ▶ **Filters Clutter:** Discards comments starting with ; (e.g., ; cost = 2) and blank lines.
- ▶ **Strips Planner Syntax:** Removes step prefixes (e.g., 0:) and outer parentheses (...).
- ▶ **Extracts Tokens:** Normalizes action name to lowercase and collects arguments:

```
"0: (place yellow_cube pot)" → ("place",  
    ["yellow_cube", "pot"])
```

2. Plan to Sequence

(pddl_plan_to_sequence) Converts the parsed action list into an executable Behavior Tree Sequence:

- ▶ **Skill Dispatch:** Uses SKILL_REGISTRY to map actions to robot skills ("pick", "place").
- ▶ **Argument Binding:** Passes simulator environment and object targets (object, asset).
- ▶ **Why memory=True?:** Ensures progress! Once the robot finishes pick, the sequence remembers it and moves to place without restarting pick on the next tick.

Compilation Pipeline Example

1. Planner Output (Fast Downward):

```
; plan generated by fast downward  
0: (pick yellow_cube)  
1: (place yellow_cube pot)  
; cost = 2
```

↓ *parsed & compiled into BT*

2. Executable PyTrees Sequence:

```
[-] pddl_plan_sequence (memory=True)  
|-- PickUp(object='yellow_cube')  
'-- PlaceInBin(object='yellow_cube',  
    asset='pot')
```

Design Rationale

Cleanly decouples **symbolic task planning** (Fast Downward) from the **physical robot control** (TaskSpaceRRT motion planning).

BT Builder: Goal Checking & Reactive Tree Generation

3. Goal-Guarded Wrapper

(wrap_with_goal_check) A plain sequence is open-loop. If an object drops, it fails blindly. We wrap it in a reactive fallback loop:

- ▶ **Pre-Check (GoalCheck_Pre)**: Verifies if goal is already met before moving. If yes → succeeds immediately (no redundant actions).
- ▶ **Selector (memory=False)**: Re-evaluates continuously on every tick for dynamic preemption.
- ▶ **Post-Check (GoalCheck_Post)**: Confirms physical completion after the sequence finishes.
- ▶ **Retry Loop (Retry)**: If the grasp slips and post-check fails, automatically retries the plan up to N times.

4. Main Entrypoint

(build_bt_from_pddl_plan) Single builder API providing two execution modes:

- ▶ **wrap_goal_check=False**: Raw Sequence (open-loop baseline).
- ▶ **wrap_goal_check=True**: Self-healing tree with retry loop.

Goal-Guarded Tree Architecture

```
[?] goal_guarded_root (Selector)
|-- [>] GoalCheck_Pre (already met?)
'-- [R] retry_until_goal (max=10)
    '-- [->] attempt (Sequence)
        |-- [->] pddl_plan_sequence
        |   |-- PickUp(yellow_cube)
        |   '-- PlaceInBin(yellow_cube->pot)
        '-- [>] GoalCheck_Post (verified?)
```

Failure Recovery Example (Cube Drop)

1. Robot picks up cube, but gripper slips mid-air.
2. Sequence finishes → GoalCheck.Post checks pot.
3. Cube is missing → returns FAILURE.
4. Retry catches failure → re-runs the plan → task completed successfully!

BT Nodes: Skills & Conditions

Action Skills (`skills.py`)

Subclasses of `py_trees.behaviour.Behaviour`:

- ▶ **Non-Blocking Stepping:** Returns `RUNNING` while the robot moves, and `SUCCESS` or `FAILURE` upon completion.
- ▶ **TaskSpaceRRT Integration:** Computes collision-free $SE(3)$ end-effector trajectories dynamically for the 7-DOF Panda arm.
- ▶ **Operational Primitives:**
 - ▶ `MotionPlanningPickUp`: Approach → Grasp → Lift.
 - ▶ `MotionPlanningPlaceInBin`: Transit → Lower → Release → Retreat.

Condition Nodes (`conditions.py`)

Dual verification paradigm:

- ▶ **PDDLGoalCheck (Symbolic Grounding):**
 - ▶ Parses goal formulas from `problem.pddl`.
 - ▶ Evaluates simulator predicates deterministically (`is_in_bin`, `is_grasped`).
 - ▶ Fast, deterministic, and zero-latency.
- ▶ **VisualCheck (VLM Inspection):**
 - ▶ Adopts Wake et al. self-prompting.
 - ▶ Queries VLM (`vlm.query_fn`) on camera frames for open-world semantic validation.

Reactive Execution & Runtime Tree

Runtime Execution Flow

1. **Initial Check:** GoalCheck_Pre runs. If goal holds, the tree returns SUCCESS immediately (no redundant actions).
2. **Execution:** On failure, the Fallback ticks `retry_until_goal` → `pddl_plan_sequence`.
3. **Closed-Loop Motion:** Each skill executes collision-free RRT trajectories at 20 Hz.
4. **Verification & Recovery:** GoalCheck_Post validates final state. If an object is dropped, the Retry decorator triggers re-execution.

Generated Behavior Tree Structure

```
[?] goal_guarded_root (Selector)
|-- [>] GoalCheck_Pre
'-- [R] retry_until_goal (max=3)
    '-- [->] attempt (Sequence)
        |-- [->] pddl_plan_sequence
        |   |-- PickUp(yellow_cube)
        |   |-- PlaceInBin(yellow_cube->pot)
        |   |-- PickUp(red_can)
        |   '-- PlaceInBin(red_can->bin)
        '-- [>] GoalCheck_Post
```